

SE2052 – Programming Paradigms

Y2 S2 – BSc (Hons) in Computer Science

Take-Home Assignment

Duration: ~ 16 hours

Submission Deadline: 04 – 05 -2026

Total Marks: 100

TaskLang++: A Domain-Specific Language for Task Scheduling and Automation

Modern systems rely heavily on task scheduling and automation, from simple reminders to complex workflows (e.g., CI/CD pipelines, cron jobs, smart assistants). General-purpose languages can express these, but they are often verbose and error-prone. In this assignment, you will design a Domain-Specific Language (DSL) called TaskLang++ to define:

- Tasks
- Time-based scheduling
- Dependencies between tasks
- Conditional execution

You will formally define the syntax using BNF or EBNF and demonstrate how your DSL simplifies task specification. Then implement a lexer using Lex or Flex and a parser using Yacc or Bison.

Task 1: Domain Understanding & DSL Scope

Define the **scope of your Task Scheduling DSL**.

- What types of tasks are supported? (e.g., email, backup, script execution)
- Scheduling mechanisms:
 - Time-based (daily, weekly, specific time)
 - Event-based (optional)

- Dependencies (e.g., task A must complete before task B)
- Constraints or assumptions

Task 2: Language Design and Define the Formal Grammar

Define the formal grammar for TaskLang++. Grammar needs to be unambiguous.

✦ Your DSL should support at least:

- Task definition
- Scheduling (AT, EVERY, AFTER, etc.)
- Dependencies (BEFORE, AFTER, DEPENDS ON)
- Optional conditions (IF)

Following are some tokens that can be used in TaskLang++

Token Name	Lexeme	Description
TASK	TASK	Begins a task definition
RUN	RUN	Specifies command execution
EVERY	EVERY	Recurring schedule keyword
DAY	DAY	Used with EVERY
AT	AT	Time-based execution
AFTER	AFTER	Dependency specification
IF	IF	Conditional execution
SUCCESS	success	Condition keyword

Task 3: Implement lexer and parser.

Task 4: Validate the parser. Test your implementation using both valid and invalid programs with appropriate output messages (successful parsing or failures)

📌 Example TaskLang++ codes could be as below:

1. A simple daily recurring scheduled task:

```
TASK dailyReport {  
    RUN "report.py"  
    EVERY DAY AT 06:00  
}
```

2. Multi-step workflow chained with conditions:

```
TASK backupDB {  
    RUN "backup.sh"  
    EVERY DAY AT 02:00  
}  
  
TASK sendReport {  
    RUN "report.py"  
    AFTER backupDB  
    IF success  
}  
  
TASK cleanup {  
    RUN "cleanup.sh"  
    EVERY WEEK ON SUNDAY AT 03:00  
}
```

You need to cover different scenarios of task combinations such as routine tasks such as daily schedules, tasks with conditions, task dependencies, how to deal with circular dependencies (if any), etc..

Expected Output

```
Parsing TaskLang++ input...
```

```
--- EXECUTION START ---
```

```
Executing Task: dailyReport
```

```
  Script: "report.py"
```

```
  Schedule: EVERY DAY AT 06:00
```

```
--- EXECUTION COMPLETE ---
```

```
Parsing TaskLang++ input...
```

```
--- EXECUTION START ---
```

```
Executing Task: backupDB
```

```
  Script: "backup.sh"
```

```
  Schedule: EVERY DAY AT 02:00
```

```
Executing Task: sendReport
```

```
  Script: "report.py"
```

```
  Schedule:
```

```
  Depends on: backupDB
```

```
  Condition: success
```

```
Executing Task: cleanup
```

```
  Script: "cleanup.sh"
```

```
  Schedule: AT 03:00
```

```
--- EXECUTION COMPLETE ---
```

Deliverables:**Report (PDF)**

1. Domain & DSL Design (scope of your Task Scheduling DSL as indicated in task 1)
 2. Final Grammar (EBNF + BNF as indicated in task 2). Clearly defined terminals and non-terminals.
 3. Sample Programs
 4. Reflection
-

Code Submission (as a zip file need to be uploaded separately)

- lexer.l
- parser.y
- Makefile or compile instructions
- Sample input files

Marking Rubric

Component	Excellent (A)	Good (B)	Satisfactory (C)	Poor (D/F)
DSL Design (10 marks)	Clear, realistic, well-justified DSL. Clear, realistic task scheduling domain; well-justified need for DSL; expressive and consistent syntax design	Reasonable design, minor issues, minor inconsistencies or limited justification	Basic domain; limited features; syntax somewhat unclear	Unclear or weak design, poor syntax
Grammar (20 marks)	Fully correct, well-structured inconsistent grammar. Covers all constructs (tasks, scheduling, dependencies, conditions), use EBNF features, proper naming, etc..	Minor errors, mostly correct and covered	Several inconsistencies, limited coverage	Incorrect/incomplete
Lexer (15 marks)	Accurate tokens, strong regex, correct prioritization of keywords and identifiers, proper handling of white spaces, Meaningful lexical error reporting	Mostly correct	Some issues	Incorrect/missing
Parser (20 marks)	Correct parsing, no conflicts, Correct mapping from BNF to Yacc, accepts valid programs, rejects invalid ones, Optional Constructs Handling, minimal conflicts, clear rules	Minor issues	Some conflicts/errors	Fails to parse
Integration and execution (10 marks)	Seamless lexer-parser, Seamless integration; tokens correctly passed; no runtime issues, Correct execution + dependencies	Minor issues, Basic execution works	Partial integration and limited simulation	No meaningful execution logic
Testing (15 marks)	Comprehensive tests covering all syntax errors, semantic errors covering all possible scenarios including edge cases, uses test automation	Good coverage	Basic tests	No testing
Reflection (10 marks)	Insightful and critical. Insightful discussion of challenges, trade-offs, improvements	Reasonable reflection	Basic reflection	Missing/weak or superficial